Université Mohamed premier
Faculté des sciences , Oujda
Département d'Informatique

Année universitaire 23/24
Filière SMI S6

Bases de données avancées

Professeur : M. MOUSSI
mohmoussi@hotmail.com

Objectifs du cours

Ce cours permet à l'étudiant de :

- Manipuler le langage SQL sous Oracle.
- Apprendre un langage de Base de données extension du langage standard SQL (PL/SQL).
- Voir les fonctionnalités d'un SGBD, en manipulant Oracle.
- Apprendre comment administrer une Base de données.

Plan du cours

- Introduction.
- Les SGBDs et leurs fonctions.
- Langage SQL (TP).
- Notion d’administration.
- Gestion des utilisateurs.
- Gestion des rôles.
- Gestion des performances : les vues et les index.
- Langage PL/SQL.

Définitions

- ***Base de données*** :
  - Collection des données :
    - Persistantes.
    - Opérationnelles.
    - Enregistrées et utilisées par des systèmes d'application (des programmes) d'une entreprise particulière.

Définitions

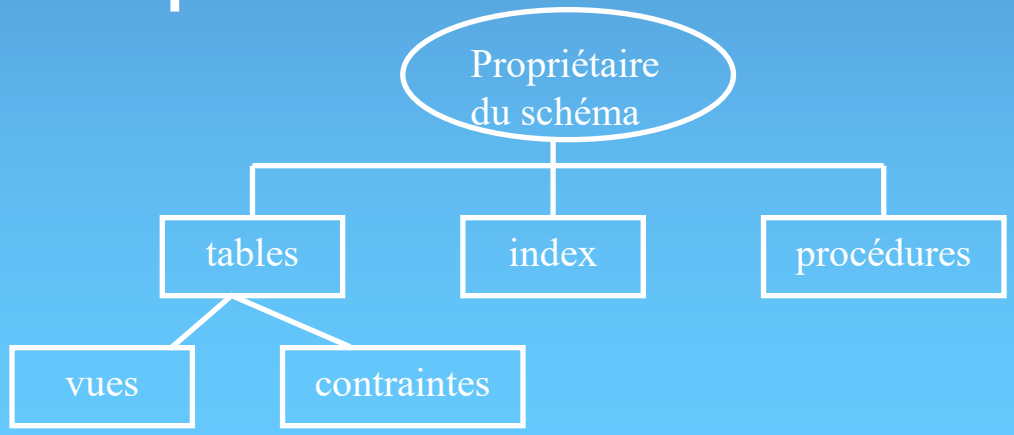
- ***Base de données*** :
  - Structurée indépendamment d'une quelconque application.
  - Cohérente.
  - De redondance minimale.
  - Peut être accessible simultanément par plusieurs utilisateurs.
  - Peut être non structuré, avec redondance, ou moins de cohérence (NoSQL).

Définitions

- ***Modèle de données :***
  - Est un Ensemble de concepts et de règles d'utilisation de ces concepts qui permettent de décrire les données d'une BD et de modéliser leur structure.
  - Est un moyen de description du monde réel.
  - Types de modèles de données principaux : Le modèle hiérarchique, le modèle réseau, le modèle relationnel, le modèle objet.

Définitions

- ***Schéma d'une base de données :***
  - Est un groupe d'objets d'une base de données (table, index, contraintes…) qui sont unis et reliés entre eux.
  - C'est une description de la base de données obtenue en employant un modèle de données.
  - **Exemple** : schéma relationnel



Définitions

- ***Système de gestion de base de données :***
  - Est un système informatique (logiciel) permettant de maintenir des données et de les rendre disponible à la demande.
  - Tout SGBD appartient à une classe associée à un modèle de données.

Composants liés à la gestion d'une base de données

- ***Les données :***
  - Données intégrées : la BD peut être vue comme une unification de plusieurs fichiers de données distincts, pas de redondance .

Exemple :

Etudiant : NumEtudiant, nom, adresse, date de naissance…

Inscription : NumEtudiant, NumModule, DateInscription…

Module : NumModule, nom_module, nbre_heure…

pour le nom d'un étudiant, il suffit de le chercher dans le fichier Etudiant.

Composants liés à la gestion d'une base de données

- Données partagées : chaque donnée de la base peut être partagée par plusieurs utilisateurs qui peuvent accéder simultanément à cette donnée.

  Exemple :

  Dans une application de gestion de la scolarité, les utilisateurs du service d'examen et du service d'inscription peuvent accéder aux informations relatives à l'étudiant dans le fichier Etudiant.

Composants liés à la gestion d'une base de données

- ***Le matériel*** : se compose des volumes de mémorisation pour le stockage des données, le processeur et la mémoire principale pour l'exécution du logiciel.
- ***Le logiciel*** : c'est le SGBD, responsable de l'exécution de toutes les requêtes des utilisateurs pour accéder à la base de données et manipuler les données.

  Exemple : Oracle, Informix, Sybase, Access,
  SQL Server…

Composants liés à la gestion d'une base de données

- ***Les intervenants*** :
  - L'administrateur de la BD : responsable sur le contrôle des données :
    - définit les types de données à Stocker dans la base,
    - délivre des autorisations d'accès à la BD,
    - décide des stratégies de maintenance et de manipulation de ces données.

Composants liés à la gestion d'une base de données

- Le programmeur de l'application : développe l'application (le programme) utilisée pour accéder à la base de données.
- L'utilisateur final : accède à la base de données à travers une interface intégrée au logiciel ou bien une application.

Fonctions fournies par un SGBD

- ***Description des données*** :
  - Description des objets (Etudiant, Enseignant, module…), leurs attributs (NumEtudiant, Nom, Adresse…).
  - Description des liens (un Etudiant est inscrit dans un module).
  - Description des contraintes éventuelles sur les objets, leurs attributs ou leurs liens.
  - *À l'aide du langage de description de données (LDD)*.

Fonctions fournies par un SGBD

- ***Manipulation des données*** :
  - Concerne les outils et les mécanismes qui permettent de faire communiquer une BD et l'utilisateur,
  - effectuer des opérations de recherche, d'insertion, de modification et de suppression sur cette base,
  - à l'aide du *langage de manipulation de données (LMD)*.

Fonctions fournies par un SGBD

- ***Assurer la cohérence des données*** :
  - Soit une information représentée d'une façon redondante dans la BD, la mise à jour de cette information dans un seul endroit rend la BD dans un état incohérent.
  - Une BD incohérente fournit des données incorrecte ou contradictoires.
  - Par l'élimination de la redondance ou bien son contrôle, le SGBD garantit la cohérence.

Fonctions fournies par un SGBD

- **Exemple** : deux étudiants avec les mêmes informations, si j'ai pas d'identifiant (numéro par exemple)?

- ***Intégrité des données*** :
  - Est relatif à la qualité de la valeur enregistrée, celle-ci doit vérifier certaines propriétés de façon à garantir l'exactitude et la cohérence des données. Ces propriétés sont appelées ***contraintes d'intégrité*.**

Fonctions fournies par un SGBD

- **Exemples** :
  - Chaque étudiant possède un numéro qui l'identifie.
  - Un étudiant ne peut suivre que six modules au plus dans un semestre.
  - La note minimale pour valider un module est 10/20.
  - On ne peut pas programmer deux cours pour le même professeur dans le même créneau.
  - On ne peut pas programmer deux cours dans la même salle dans le même créneau.
  - La durée d'une séance de cours ne peut excéder 2 heurs et ne peut être moins que 1Heure et ½.
  - …

Fonctions fournies par un SGBD

- ***Fiabilité*** :
  - Risque de perte d'informations à cause d'un incident matériel ou logiciel : erreur de programmation, panne système, coupure du courant, le disque dur qui contient la base est endommagé, ...
  - Le SGBD doit offrir la possibilité de remettre la BD dans un état opérationnel.
  - La journalisation des opérations réalisées sur la base et leur réexécution automatique en cas de besoin.

Fonctions fournies par un SGBD

- ***Les accès concurrents*** :
  - Plusieurs utilisateurs peuvent accéder aux données simultanément.
  - Le SGBD doit gérer les conflits qui peuvent être résultés de ces accès.

Exemple : Soient les deux opérations de mise à jour, notées OP1 et OP2, qui s'effectuent sur une même donnée V=100. On a deux cas :

Fonctions fournies par un SGBD

- OP1 et OP2 sont séquentielles :

OP1	OP2
X←Lire(V)	Y←Lire(V)
X←X+50	Y←Y+100
Ecrire(X, V)	Ecrire(Y, V)

Après exécution séquentielle des deux OP1 et OP2, on obtient V =250.

Fonctions fournies par un SGBD

- Exécution d'une manière concurrente :

OP1	**OP2**
X←Lire(V)	
X←X+50	
	Y←Lire(V)
Ecrire(X, V)	
	Y←Y+100
	Ecrire(Y, V)

Après exécution concurrente, on obtient V =200.

Fonctions fournies par un SGBD

- Conclusion :
  - la mise à jour de l'opération OP1 a écrasé celle de l'opération OP2. Il y'a donc une perte de mise à jour.
  - Pour éviter ce genre de problème, la technique de verrouillage est souvent utilisée lors d'accès en modification.

Fonctions fournies par un SGBD

- ***Confidentialité*** :
  - Toute information doit être protégée contre l'accès des utilisateurs non autorisés soit en écriture ou en lecture.
  - La confidentialité est assurée par :
    - Le biais de mot de passe et de gestion des droits d'accès (par utilisateur ou par groupe).
    - L'utilisation des sous-schéma.

Fonctions fournies par un SGBD

- **Exemple** :

  Dans l'application de la gestion des enseignements Les étudiants sont autorisés à consulter les noms des modules et les horaires de l'enseignement, mais ils ne sont pas autorisés à les modifier…

  Un employer est autorisé à consulter les informations d'un étudiant mais n'est pas autorisé à consulter ses notes.

Notion d'administration

- Les rôles de l'administrateur peuvent être classés en deux catégories:
  - La conception de la base de données,
  - l'organisation technique de la base de données.

Notion d'administration

- Le rôle d'un administrateur de BD :
  - Installer le SGBD et les différents produits qui lui sont associés :
    - Etudier le manuel d'installation.
    - Préparer le système d'exploitation.
    - Estimer l'espace disque nécessaire pour les produits à installer et la base de départ.
    - …etc

Notion d'administration

- Créer la BD et les objets des schémas, en suivant le modèle conceptuel.
- Faire évoluer la BD suivant les besoins des applications et l'évolution du schéma conceptuel.
- Allouer des espaces systèmes et planifier des besoins futurs.

Notion d'administration

- Gérer la sécurité de la BD:
  - Créer les utilisateurs qui pourront se connecter à la BD.
  - Définir les droits des utilisateurs en fonction de leur rôle dans la BD.
- Faire le suivi et la maintenance de la BD ainsi que la maintenance du matériel utilisé.

Notion d’administration

- Assurer l’intégrité des données et mettre en place des procédures pour faire face aux différentes défaillances qui peuvent arriver.
- Planifier l'archivage de données.
- Sauvegarder et restaurer les données :
  - Définir une stratégie de sauvegarde.
  - Adapter le rythme des sauvegardes par rapport au besoin.

Notion d'administration

- Arrêter et démarrer la base de données.
- Contrôler et optimiser les performances
- En général : il est chargé de la création, la maintenance et la sécurité de la base de données.

Gestion des utilisateurs

- Chaque base de données Oracle possède une liste d'utilisateurs valables.
- Pour pouvoir se connecter à l'instance d'une BD, un utilisateur doit utiliser un nom d'utilisateur défini dans celle-ci.

Gestion des utilisateurs

- Création de nouveaux utilisateurs:
  - Pour créer un utilisateur d'une BD, il faut l'identifier en lui donnant un nom et un mot de passe.
  - Syntaxe :

    CREATE USER nom_utilisateur

    IDENTIFIED BY mot_de_passe ;

Gestion des utilisateurs

- Plus d'options de la création:
  - *DEFAULT TABLESPACE tablespace* (par défaut, le tablespace SYSTEM)
  - *TEMPORARY TABLESPACE* tablespace (par défaut, la tablespace TEMP)
  - *QUOTA {entier[K,M] | UNLIMITED} ON tablespace.*
  - *ACCOUNT {LOCK / UNLOCK}*
  - *…*

Gestion des utilisateurs

- Modification d'un utilisateur :
  - La modification d'un utilisateur repose sur toutes les options de sa création.
  - Syntaxe :

    ALTER USER nom_utilisateur

    …
  - **Exemple** :

    CREATE USER etudiant IDENTIFIED BY et_pwd;

    ALTER USER etudiant IDENTIFIED BY new_et_pwd;

Gestion des utilisateurs

- Suppression d'un utilisateur :
  - La suppression d'un utilisateur entraîne la suppression de tous les objets de son schéma et de leurs références dans le dictionnaire de données. Par ailleurs, si l'utilisateur possède des objets dans son schéma, il faut utiliser l'option *CASCADE;*
  - Syntaxe :
  - *DROP USER nom_utilisateur [CASCADE] ;*

Langage de contrôle de données

- Quand un utilisateur est créé, il ne possède aucun privilège sur la BD, y compris celui de se connecter.
- Selon les besoins, le DBA (l'administrateur de la BD) attribue des privilèges à un utilisateur.

Langage de contrôle de données

- Il existe deux types de privilèges :
  - Les privilèges systèmes qui concernent la connexion à la BD et la gestion des objets et qui sont accordés par un utilisateur qui possède lui-même ces privilèges et celui de les accorder.
  - Les privilèges objets qui portent sur des objets existants et qui sont accordés par le propriétaire de ces objets.

Langage de contrôle de données

- La commande permettant d'accorder les privilèges systèmes est :

  GRANT { privilège système }

  [{ privilège système },…]

  to

  { utilisateur } [{ utilisateur },…]

  [WITH ADMIN OPTION].

  - L'option *WITH ADMIN OPTION* permet au bénéficiaire des privilèges d'accorder lui-même ces privilèges à d'autre utilisateurs

Langage de contrôle de données

- Pour désigner tous les utilisateurs, on utilise l'option *PUBLIC*

- Exemples des privilèges systèmes :
  - *CREATE SESSION :* se connecter à la BD.
  - *CREATE TABLE* : créer des tables
  - *CREATE ANY TABLE* : créer des tables sur tous les schémas.
  - *CREATE VIEW* : créer des vues
  - *CREATE TABLESPACE* : créer des tablespaces
  - *CREATE USER :* créer des utilisateurs.

Langage de contrôle de données

- *ALTER TABLESPACE* : modifier des tablespaces
- *DROP TABLESPACE* : supprimer des tablespaces
- *ALTER ANY TABLE.*
- *DROP ANY TABLE.*
- *ALTER USER.*
- *DROP USER.*
- **Exemple** :

GRANT CREATE SESSION, CREATE TABLE,
CREATE VIEW TO utilisateur1;

Langage de contrôle de données

- La commande permettant de retirer un privilège système à un utilisateur est *REVOKE.*
  - **Exemple**:
    - *REVOKE CREATE VIEW FROM utilisateur1;*

Langage de contrôle de données

- Quand un utilisateur crée des objets dans le schéma d'un autre utilisateur, il ne possède pas tous les droits sur ces objets.
- Quand un utilisateur crée des objets dans son schéma, il possède tous les droits sur ces objets.
- Il a, lui seul, la possibilité d'accorder des privilèges sur ses objets à d'autres utilisateurs, ainsi que de les retirer.

Langage de contrôle de données

- Syntaxe :

  GRANT

  { privilège objet [, privilège objet,…] } | ALL }

  TO utilisateur [, utilisateur…]

  [WITH GRANT OPTION]

  - L'option *WITH GRANT OPTION* permet la transmission des privilèges objets.

Langage de contrôle de données

- Exemples de privilèges objets :
  - Sur une table :
    - *SELECT, DELETE, INSERT, UPDATE*
  - Sur une vue :
    - *DELETE, INSERT,SELECT, UPDATE.*
  - Sur les procédures stockées :
    - *EXECUTE*

Langage de contrôle de données

- **Exemple** :

  GRANT SELECT, INSERT, UPDATE(libelle,prixunit) ON Produit TO utilisateur1;

- Le propriétaire des objets peut retirer les privilèges accordés sur les objets à l'aide de l'instruction *REVOKE.*
  - **Exemple** :

    REVOKE INSERT ON produit FROM utilisateur1;

Gestion des rôles

- Un rôle est un regroupement de privilèges système et objet et qui peut être attribué à un utilisateur ou à un autre rôle.
- L’attribution d’un rôle à un utilisateur lui permet de posséder tous les privilèges regroupés dans ce rôle.

Gestion des rôles

- Les rôles permettent de simplifier la gestion des droits des utilisateurs.
- Plusieurs rôles peuvent être attribués à un utilisateur.

Gestion des rôles

- Création d'un rôle :
  - Pour créer un rôle, il faut avoir le privilège système *CREATE ROLE*
  - Syntaxe :

CREATE ROLE nom_role

IDENTIFIED BY mot_de_passe;

Gestion des rôles

- Attribution d'un privilège à un rôle :
  - Syntaxe :
    - *GRANT nom_privilège [,nom_privilège...]*
      TO nom_rôle [,nom_rôle...]
      [WITH ADMIN OPTION] ;
    - *GRANT { nom_privilège[, nom_privilège…] | ALL }*
      ON [nom_schéma.]nom_objet
      TO nom_rôle [,nom_rôle...] ;

Gestion des rôles

- Attribution d'un privilège à un rôle :
  - Pour attribuer un privilège système à un rôle Il faut :
    - Avoir *WITH ADMIN OPTION*
    - Ou avoir le privilège système *GRANT ANY PRIVILEGE.*
  - Pour attribuer un privilège objet il faut:
    - Etre propriétaire
    - Ou avoir WITH GRANT OPTION
    - OU avoir le privilège système *ANY OBJECT PRIVILEGE*

Gestion des rôles

- **Exemple** :

GRANT CREATE SESSION, CREATE TABLE

TO SMI22_role;

GRANT SELECT, UPDATE (libelle), INSERT

ON Produit

TO SMI22_role;

Gestion des rôles

- L'instruction REVOKE permet de retirer un privilège système ou objet d'un rôle.
  - Syntaxe :
    - *REVOKE nom_privilège [,nom_privilège...]*
      FROM nom_rôle [,nom_rôle...] ;
    - *REVOKE { nom_privilège[,nom_privilège…] | ALL }*
      ON [nom_schéma.]nom_objet
      FROM nom_rôle [,nom_rôle...] ;

Gestion des rôles

- L'instruction REVOKE
  - Soit un privilège attribué à un rôle, ce rôle est attribué à un utilisateur. Possibilité de retirer le privilège de l'utilisateur?

Gestion des rôles

- Attribution d'un rôle à un utilisateur ou à un rôle :
  - Syntaxe :
    - *GRANT nom_role [,nom_role...]*
      TO {nom_utilisateur | PUBLIC |nom_rôle } [,…]
      [WITH ADMIN OPTION] ;
- La commande *REVOKE* permet de retirer un rôle.

  REVOKE nom_rôle [,...] FROM { nom_utilisateur | PUBLIC | nom_rôle } [,...] ;

Gestion des rôles

- Attribution d'un rôle à un utilisateur ou à un rôle :
  - Pour attribuer un rôle à un utilisateur, il faut avoir reçu ce rôle avec l'option *WITH ADMIN OPTION ;*
  - La clause *WITH ADMIN OPTION* donne aussi le droit de modifier.
  - Pour retirer un role, il faut avoir reçu ce rôle avec l'option *WITH ADMIN OPTION* ou avoir le privilège *GRANT ANY ROLE.*

Gestion des rôles

- Suppression d'un rôle :
  - Il faut avoir reçu un rôle avec l'option *WITH ADMIN OPTION* pour pouvoir le supprimer.
  - Syntaxe :

    DROP ROLE nom_role ;

Les vues

- Une vue est une table virtuelle, ses données ne sont pas stockées dans une table de la base de données.
- Peut être créée sur une ou plusieurs tables.
- Est une fenêtre sur les tables d’un schéma.
- Est une vision partielle ou particulière des données d'une ou plusieurs tables de la BD.

Les vues

- La définition d'une vue est donnée par un *SELECT* qui indique les données de la base qui seront vues.
- Une vue n’existe pas physiquement et n’induit aucun stockage.
- Une vue est dynamique, son résultat est réévalué à chaque fois que l’on accède à la vue.

Les vues

- La vue permet de
  - regrouper des informations au sein d'une même entité,
  - limiter l'accès à la base de données,
  - sécuriser l'accès aux données : une partie de l'information est invisible pour l'utilisateur de la vue,
  - faciliter l'accès aux données : surtout pour des informations constituées par des jointures.

Les vues

- A travers les vues, les utilisateurs pourront consulter, ou modifier la base avec les restrictions suivantes :
  - Construite sur une seule table.
  - Possible sur une vue comportant une jointure si la modification ne concerne qu'une seule table de la jointure et que la clé primaire de cette table soit préservée par la vue.

Les vues

- Pas de *GROUP BY, HAVING* ou *DISTINCT.*
- Les colonnes résultats doivent être des colonnes réelles de la table (non calculées).
- Les colonnes ne peuvent être dérivées d'une fonction d'agrégation : *AVG, COUNT, SUM, MIN, MAX*

Les vues

- Syntaxe de création d'une vue :

  CREATE [OR REPLACE] [FORCE|NOFORCE] VIEW Nom_Vue

  [(colonne1,…)]

  AS SELECT …

  [WITH CHECK OPTION | WITH READ ONLY];

  - *WITH CHECK OPTION* : impose que toutes les instructions de modification de données exécutées sur la vue respectent le critère défini dans la requête *SELECT*.

Les vues

- *WITH READ ONLY :* interdit toute insertion, modification, suppression de données à travers la vue.
- *NOFORCE VIEW* : permet de créer des vues lorsque la table ou les tables utilisées existent et le propriétaire du schéma contenant la vue possède des privilèges sur ces tables. C'est l'option par défaut.

Les vues

- *FORCE VIEW :* permet de créer la vue sans se soucier de l'existence des tables qu'elle référence ou des privilèges adéquats sur les tables.
- *OR REPLACE :* permet de modifier la définition d'une vue sans devoir la supprimer, la recréer ou affecter les privilèges des objets déjà définis pour la vue courante.

Les vues

- Par défaut, les colonnes de la vue ont pour nom les noms des colonnes résultats du *SELECT*.
- Si certaine colonne résultat du *SELECT* est une expression sans nom, il faut spécifier le nom de la colonne de la vue.

Les vues

- Dans le cas d'utilisation de la clause de restriction *with,* l'instruction *SELECT* peut contenir toutes les clauses d'un *SELECT*, sauf la clause *ORDER BY.*

Les vues

- On distingue les vues simples et les vues complexes.
  - Une vue simple :
    - Est dérivée des données d'une seule table,
    - ne contient ni fonctions ni groupes de données,
    - permet d'exécuter des opérations du *LMD*.

Les vues

- Une vue complexe :
  - Est dérivée des données de plusieurs tables.
  - Contient des fonctions ou des groupes de données.
  - Ne permet pas toujours d'exécuter des opérations du *LMD*.

Les vues

- Exemples :

```
CREATE VIEW  produitview
 AS SELECT  *    FROM produit
 WHERE      prix_uni <= 20;
```

```
Insert into produitview values('P2', 'S623',10, 30)
Insert into produitview values('P3', 'smi23',40, 30)
```

Les vues

- Exemples :

```
CREATE VIEW  produitview
 AS SELECT  *  FROM  produit
 WHERE                    prix_uni <= 20
 WITH CHECK OPTION;
```

```
Insert into produitview values('P4', 'S623',10, 30)
Insert into produitview values('P5', 'smi23',40, 30)
```

Les vues

```
CREATE VIEW  nbreProduit_cde
AS SELECT Num_cde, count(ref_produit)
FROM ligne_commande
Group by num_cde
```

Les vues

```
CREATE VIEW  prod_cde
AS SELECT p.ref_produit,  p.libelle, l.quantite
FROM produit p, ligne_commande l
WHERE p.ref_produit=l.ref_produit;
```

Les vues

update prod_cde
Set quantite=quantite+1;

update prod_cde
Set quantite=quantite+1,
Libelle='modification'
Where ref_produit='p1'

Les vues

- L'ordre *DROP VIEW* permet de supprimer une vue.
- La suppression d'une vue n'a aucun effet sur les tables référencées.
- Seul le propriétaire ou un utilisateur ayant le privilège *DROP ANY VIEW* peut supprimer une vue.

Les vues

- Actualiser une vue
  - Les enregistrements de la vue s'actualise automatiquement.
  - Si la structure d'une des tables appelées par la vue change, cette dernière renvoie une erreur.
  - Pour la mettre à jour :

    ALTER VIEW nom_vue COMPILE;

Les index

- Un index, dans le domaine bibliographique, permet de lister les mots-clés importants abordés dans un ouvrage et d'indiquer les pages où le mot est mentionné.
- Un lecteur qui recherche une thématique spécifique peut se baser sur cet index pour trouver les pages qui abordent le sujet.

Les index

- Ainsi un index est un objet permettant à un utilisateur de gagner le temps en accédant facilement à l'information recherchée.

Les index

- Un index, dans une base de données se base sur le même principe qu'un index dans un livre.
- Avec un index placé sur une ou plusieurs colonnes le système d'une base de données peut rechercher les données d'abord sur l'index et s'il trouve ce qu'il cherche il saura plus rapidement où se trouve les enregistrements concernés.

Les index

- La création des index a toutefois ces inconvénients car cela occupe de l'espace supplémentaire dans la base de données. Par ailleurs, l'insertion de données est plus longue car les index sont mis à jour à chaque fois que des données sont insérées.

Les index

- Par conséquent, il faut que la création d'index soit justifiée et que les colonnes sur lesquelles il porte soient judicieusement choisies (de telle façon à minimiser les doublons).
- Généralement, un index pourra être utilisé dans les requêtes utilisant les clauses *WHERE*, *GROUP BY* ou *ORDER BY*.

Les index

- Lorsqu'une base de données possède un grand nombre d'enregistrements (exemple : plusieurs milliers ou plusieurs millions de lignes) un index permet de gagner un temps précieux pour la lecture de données.
- Les index réduisent les temps de réponse en lecture seule, mais pénalisent les performances en modification (*INSERT , UPDATE , DELETE).*

Les index

- Sous Oracle, avec la contrainte *UNIQUE* ou *PRIMARY KEY*, un index unique est automatiquement créé.
- Un index peut être créé sur une seule colonne ou sur plusieurs colonnes.

Les index

- Syntaxe de création d'index :

  CREATE [UNIQUE] INDEX nom_index ON Nom_table (colonne1, colonne2, ..);

- La clause *UNIQUE* permet de spécifier qu'une ou plusieurs colonnes doivent contenir des valeurs uniques à chaque enregistrement

Les index

- Il est possible de créer plusieurs index indépendants sur une même table.
- ***Exemple*** :

  CREATE UNIQUE INDEX ix_produit

  ON produit (libelle);

Cet index empêche la saisie des noms en doublon dans la colonne *nom_etud*.

Les index

Pour supprimer un index :

DROP INDEX nom_index [ON table];

- Le nom de la table doit être précisé si on veut supprimer un index d'une table d'un autre utilisateur alors qu'on possède un index du même nom.
- Un index est automatiquement supprimé avec la suppression de la table à laquelle il est lié.

Langage PL-SQL

- Plan :
  - Introduction.
  - Les variables.
  - Les structures de contrôle.
  - Les curseurs.
  - Gestion des erreurs (les exceptions).
  - Fonctions et procédures.

Introduction

- PL/SQL(Procedural Language/Structured Query Language).
  - Représente la contribution d'Oracle corporation au monde de la programmation.
  - Il s'agit d'une extension procédurale au langage SQL.
  - Langage 4ème génération.
  - Le PL/SQL est apparu initialement avec la version 6 d'Oracle en 1988 (Oracle 12c en 2013 et à partir de 2018 Oracle 18 (par année)).
  - Sert à lire des données, à remplir des BDs, à créer des objets stockés .

Introduction

- PL/SQL : langage de programmation d'Oracle
  - Oracle représente bien plus qu'une simple base de données,
  - il inclut aussi un moteur pour de nombreux langages, parmi lesquels :
    - Java avec JVM intégré.
    - PL/SQL.
  - Le code peut être stocké dans la BD puis être exécuté sans qu'aucun produit supplémentaire ne soit nécessaire.

Introduction

- PL/SQL est utilisé dans de nombreux produits Oracle :
  - Le serveur Oracle,Oracle forms, Oracle reports, Oracle warehouse builder, oracle portal …
- Les programmes PL/SQL peuvent être appelés à partir des environnements de développement Oracle suivants :
  - SQL*Plus, Oracle Entreprise Manager, les pré-compilateurs Oracle (Pro*C, Pro*Cobol, etc.)…

Introduction

- Jeu de caractère de PL/SQL :
  - Les caractères supportés sont :
    - Les lettres majuscules et minuscules,
    - les chiffres entre 0 et 9,
    - les symboles suivants :

    ( ) + - * / < > = ! ~ ; : . – ` @ % , " ' # ^ & _ | { } ? [ ]

Introduction

- Les opérateurs arithmétiques :

Opérateur	Signification
**	Exponentiation
*	multiplication
/	Division
+ -	Addition, soustraction
||	Concaténation
:=	Assignation
=	Égale
<> *Ou* !=	Non égale
> <	Supérieur, inférieur à
>= <=	Supérieur ou égale inférieur ou égale

Introduction

- Structure d'un programme PL/SQL(bloc PL/SQL) :
  - Il y'a de la place pour des variables, pour le code et la logique du programme, des instructions de gestion d'erreurs.
  - Structure de base d'un programme PL/SQL :

```
DECLARE
   --déclaration des constantes et des variables
BEGIN
   --code du programme (commandes et instructions)
EXCEPTION
         --code de gestion des erreurs
END;
```

Introduction

- Remarque :
  - Commentaires :
    - Délimiteurs multi-lignes début et fin /* et */.
    - Délimiteurs mono-ligne de début --
  - Les sections déclaration des variables et exception sont facultatives,
  - la section obligatoire est celle débutée par le mot clé *BEGIN* et terminée par le mot clé *END.*

Les variables PL/SQL

- Sont déclarées dans la section de la déclaration d'un programme PL/SQL.
- Une déclaration
  - alloue de l'espace de stockage pour une valeur,
  - spécifie son type de données et nomme l'emplacement de stockage pour y référer au sein du programme.
- Les types de données les plus utilisés sont :

  VARCHAR2, *NUMBER*, *DATE*, *BOOLEAN*, *LONG*,

Les variables PL/SQL

- Syntaxe de déclaration :

Nom_var [CONSTANT] type_var [[NOT NULL] := expression] ;

- Syntaxe d'affectation d'une valeur à une variable dans la section d'exécution

Nom_variable :=valeur ;

Les variables PL/SQL

- **Exemple :**

```
DECLARE
        v_couner NUMBER := 0 ; --initialise la variable à 0
        v_today DATE := sysdate ; --assigne la date système à la variable
       v_name VARCHAR2(50) ;--déclaration de la variable
    BEGIN
      v_counter := v_counter+1 ;--ajoute 1 à la valeur courante du compteur
       v_name := 'mohammed' ; -- assigne une valeur à la variable
    END;
```

Les variables PL/SQL

Type de données	Description
BINARY_INTEGER	Utilisé pour stocker des entiers signés compris dans l'intervalle [-214783647..214783647]
PLS_INTEGER	C'est un sous type de BINARY_INTEGER c.à.d. le même intervalle. Les opérations sur ce type sont plus rapide que NUMBER et BINARY_INTEGER (database<10G sont plus rapide a partir de 10G c'est la même chose BINARY ou PLS)
CHAR(l)	Utilisé pour stocker des données de type (chaîne) caractère, où l représente la longueur (taille maximale).
VARCHAR(l)	Champ de longueur variable utilisé pour stocker des données de type (chaîne) caractère, où l représente la longueur (taille maximale).

Les variables PL/SQL

Type de données	Description
NUMBER(l,d)	Champ de longueur variable utilisé pour stocker des données de type numérique, où l représente la longueur (le nombre total des chiffres) et d le nombre de décimales (chiffres après la virgule).
RAW(l)	Champ de longueur variable utilisé pour stocker des données binaires, où l représente la longueur.
DATE	Champ de longueur fixe utilisé pour stocker des dates. Par défaut oracle affiche la date au format DD-Mon-YY (14-jan-2011)

Les variables PL/SQL

Type de données	Description
LONG (l)	Champ de longueur variable utilisé pour stocker jusqu'à 2G de données de type (chaîne de caractères).
BOOLEAN	Pour stocker des valeurs logiques : TRUE, FALSE ou NULL (en PL/SQL on n'a pas TRUE=1 et FALSE=0).

Les variables PL/SQL

- Variable référencée à une colonne d'une table de la base :

  Nom_variable table.colonne%TYPE;

  - **Exemple**

```
DECLARE
    ref_pdt produit.Ref_produit%TYPE;
BEGIN
    ref_pdt := ' p1 ';
END ;
```

Les variables PL/SQL

- Variable référencée à une table de la base :

  Nom_variable table%ROWTYPE;

  - **Exemple**

```
DECLARE
   un_produit produit%ROWTYPE;
```

  - Par référence aux champs :

```
BEGIN
 un_produit.Ref_produit='p1';
un_produit.libelle=' survetement sport';
un_produit.prix_unit=500 DH;
END;
```

Les variables PL/SQL

- Le nom d'une variable :
  - Est composé de lettre, chiffre, $,_ ou #.
  - Ne peut pas excéder 30 caractères.
  - Ne doit pas contenir des mots réservés.
  - Doit commencer par un caractère alphabétique.
- La valeur d'une constante ne peut pas être modifiée.
- On ne peut pas déclarer plus qu'une variable sur la même ligne.

Les variables PL/SQL

- Création de sous type :

  SUBTYPE nom_sous_type IS TYPE;

  - **Exemples :**

```
declare
subtype entier_court is binary_integer range 1..10;
x entier_court;
begin
x:=11;
end;
```

Les variables PL/SQL

```
declare
subtype entier_nonul is binary_integer NOT NULL;
y   entier_nonul:=1;
begin
y:=2;
end;
```

Les variables PL/SQL

- Affichage :

  DBMS_OUTPUT.PUT_LINE

  - Est une procédure prédéfinie.
  - Permet d'afficher des données.
  - Elle est désactivée par défaut.
  - Doit être activée dans SQL*Plus en utilisant

    SET SERVEROUTPUT ON

Les structures de contrôle

- On dispose des structures suivantes:
  - Les structures avec *IF.*
  - Les expressions *CASE.*
  - Les boucles.

Les structures de contrôle

- Les structures avec *IF* :
  - *IF-THEN*
    - Sert à tester des conditions simples.
    - Si la condition est évaluée comme étant *TRUE,* une ou plusieurs lignes de code sont exécutées.
    - Si elle est évaluée comme étant *FALSE,* aucune action n'a lieu.
    - Il est possible d'imbriquer des structures *IF-THEN*

Les structures de contrôle

– **Exemple1** :

IF qte_stock =0 THEN

DBMS_OUTPUT.PUT_LINE(‘ produit inexistant‘);

END IF ;

Ou bien

IF NOT (qte_stock != 0) THEN

DBMS_OUTPUT.PUT_LINE(‘ produit inexistant‘);

END IF ;

Les structures de contrôle

– **Exemple2** :

```
IF qte >  10 THEN
    IF  Ref_produit ='Ap1' THEN
        qte:=qte+1 ;
    END IF ;
END IF ;
```

Les structures de contrôle

- *IF-THEN-ELSE*
  - Lorsque la condition est évaluée comme étant *FALSE*, l'instruction qui suit la clause *ELSE* est exécutée.
  - Une instruction *IF* ne peut contenir qu'une seule *ELSE*
  - **Exemple** :

```
IF qte_stock =0 THEN
    DBMS_OUTPUT.PUT_LINE(' produit inexistant' );
ELSE
    DBMS_OUTPUT.PUT_LINE('produit existant' );
END IF ;
```

Les structures de contrôle

- *IF-THEN-ELSIF*
  - Offre la possibilité de tester une autre condition lorsque la première est évaluée comme étant *FALSE*.
  - Convenable lorsqu'on souhaite tester plusieurs conditions sans avoir à imbriquer des instructions *IF.*
  - Supporte un nombre illimité de clause *ELSIF*.

Les structures de contrôle

– **Exemple** :

```
IF  Ref_produit ='Ap1' THEN
      qte:=qte+1 ;
ELSIF  Ref_produit ='Ap2' THEN
      qte:=qte+2 ;
ELSE
      qte:=qte+3  ;
END IF;
```

– **Remarque** : Seule l'instruction *IF* doit être accompagnée des mots clés *END IF*, pas là où la clause *ELSIF*.

Les structures de contrôle

- Les expressions *CASE :*
  - *A été introduit avec oracle 9i,*
  - *Syntaxe :*

```
CASE sélecteur
WHEN expression1  THEN résultat1
WHEN expression2  THEN résultat2
WHEN expression3  THEN résultat3
ELSE resultat
END ;
```

Les structures de contrôle

- La première clause *WHEN* qui correspond à la valeur du sélecteur détermine la valeur du résultat, et les clauses *WHEN* suivantes ne sont pas évaluées.
- Une instruction *CASE* peut contenir un nombre illimité d'expressions.

Les structures de contrôle

– **Exemple** (ce programme affiche ou):

```
DECLARE
   Val  VARCHAR2 (100) ;
    City VARCHAR2 (20) :='oujda' ;
BEGIN
Val := CASE  city
                WHEN  'oujda' THEN 'ou'
                WHEN  'oujda2' THEN 'ou'2
                WHEN  'oujda3' THEN 'ou3'
ELSE 'no one'
          END ;
DBMS_OUTPUT.PUT_LINE(val) ; -- affichage à l'écran
END ;
```

Les structures de contrôle

- Les boucles
  - Une boucle permet d'exécuter à répétition une série d'instructions.
  - Trois types de boucles :
    - *LOOP*
    - *WHILE-LOOP*
    - *FOR-LOOP*

Les structures de contrôle

- La boucle *LOOP*
  - Le plus simple,
  - débute par une instruction *LOOP* ,
  - se termine par les mots clés E*ND LOOP*.
  - Syntaxe :

```
LOOP
   Instructions exécutables ;
END LOOP ;
```

  - Pour que cette boucle puisse prendre fin, on doit inclure une instruction *EXIT.*

Les structures de contrôle

– **Exemple1** :

```
loop
if m > 10 then exit;
else
dbms_output.put_line(m);
m:=m+1;
end if;
end loop;
```

Les structures de contrôle

– Exemple2 :

```
DECLARE
    nombre NUMBER(2) :=1 ;
BEGIN
  LOOP
        DBMS_OUTPUT.PUT_LINE(nombre) ;
       nombre := nombre+1 ;
EXIT WHEN nombre > 10 ;
END LOOP ;
END;
```

Les structures de contrôle

- La boucle *WHILE-LOOP*
  - SON Exécution se poursuit tant que la condition qu'elle définit est *TRUE*.
  - Dés que la condition est évaluée comme étant *FALSE*, la boucle se termine.,
  - Syntaxe :

```
WHILE  condition
  LOOP
     Instructions exécutables ;
  END LOOP ;
```

Les structures de contrôle

– Exemple :

```
WHILE nombre< 10
  LOOP
      DBMS_OUTPUT.PUT_LINE(nombre) ;
      nombre := nombre+1;
  END LOOP ;
```

Les structures de contrôle

- La boucle *FOR-LOOP*
  - L'une des boucles les plus utilisées en PL/SQL.
  - Son avantage est qu'on peut contrôler le nombre de fois où elle sera exécutée.
  - Syntaxe :

```
FOR variable IN val_debut .. val_fin
  LOOP
    Instructions ;
  END LOOP ;
```

Les structures de contrôle

- On peut aussi imposer que le comptage se fasse en sens inverse au moyen de la clause *REVERSE.*
- **Exemple** :

```
DECLARE
        v_counter number ;
BEGIN
     FOR v_counter IN REVERSE 1..5
     LOOP
          DBMS_OUTPUT.PUT_LINE(v_counter) ;
    END LOOP ;
END ;
```

Les structures de contrôle

- La valeur de sortie de la boucle peut être obtenue à partir de la base de données ou d'une autre source.
- **Exemple** :

```
DECLARE
        Prod_count NUMBER ;
        i number ; --variable de compteur
BEGIN
--récupèrer le nombre des produits dans variable etud_count
    SELECT COUNT(*) INTO Prod_count
     FROM produit;
    FOR i IN 1..Prod_count LOOP
      DBMS_OUTPUT.PUT_LINE('produit'|| i) ;
    END LOOP ;
   END ;
```

Les curseurs

- Le moyen le plus simple d'inclure des données dans un programme.
- Structure de données permettant de stocker le résultat d'une requête qui retourne plusieurs lignes.
- Lorsqu'on utilise un ordre SQL, un curseur implicite est automatiquement ouvert pour le gérer.
- Leur utilisation peut ralentir le programme.
- Une méthode plus efficace consiste à utiliser un curseur explicite.

Les curseurs

- Un curseur est défini dans la section de déclaration d'un programme.
- Il reçoit un nom à l'instar d'une variable.
- Oracle peut ainsi préparer à l'avance le traitement de l'instruction, ce qui permet au programme d'exploiter au mieux la mémoire.

Les curseurs

- Déclaration d'un curseur :
  - Syntaxe

    CURSOR nom_curseur IS requêteSQL;
  - **Exemple**

```
DECLARE
  CURSOR get_produit IS
 SELECT Ref_produi, libelle, prix_unit
 FROM  produit
  WHERE  qte_stock<=10;
BEGIN
…
END ;
```

Les curseurs

- Ouverture d'un curseur :
  - Syntaxe

    OPEN Nom_curseur ;
  - Il n'est pas permis d'ouvrir un curseur déjà ouvert.
  - **Exemple**

```
DECLARE
  CURSOR get_produit IS
 SELECT Ref_produi, libelle, prix_unit
 FROM  produit
  WHERE  qte_stock<=10;
BEGIN
  OPEN get_produit;
  …
END ;
```

Les curseurs

- Fermeture d'un curseur :
  - S'effectue dans les sections *BEGIN* ou *EXCEPTION.*
  - Libère les ressources utilisées par le curseur
  - Syntaxe :

    CLOSE Nom_curseur ;

  - La non fermeture du curseur peut provoquer des problèmes de mémoire ou bien peut donner des résultats non attendus.

Les curseurs

- Traitement des lignes d'un curseur :
  - Il est nécessaire de déclarer et d'ouvrir le curseur avant son utilisation.
  - **Syntaxe**
    - *FETCH Nom_curseur INTO Liste_de_variables;*
    - *FETCH Nom_curseur INTO variables_type_enregistrement;*

Les curseurs

- **Exemple :**

```
DECLARE
    prod_count  NUMBER ;
    i  number ;
--Déclarer le curseur explicite pour la sélection
      CURSOR get_prod_data IS SELECT COUNT (*) FROM
      produit;
BEGIN
--ouvrir le curseur
    OPEN  get_prod_data ;
--placer le résultat dans une variable
    FETCH get_prod_data INTO prod_count ;
    FOR i IN 1..prod_count  LOOP
        Dbms_output.put_line('produit'|| i) ;
    END LOOP ;
--fermer le curseur
    CLOSE  get_prod_data ;
END ;
```

Les curseurs

- **Remarques** :
  - Dans cet exemple, oracle extrait seulement une ligne de la base de données.
  - Pour en récupérer davantage, il est possible d'associer un curseur explicite à une boucle.

Les curseurs

- La structure *CURSOR-FOR-LOOP* :
  - Permet d'associer un curseur à une boucle.
  - Permet d'extraire plusieurs lignes de la base de données,
  - plus simple à programmer.
    - Il n'est pas nécessaire d'ouvrir et de fermer le curseur,
    - c'est oracle qui s'en charge dans la boucle.

Les curseurs

- Exemple :

```
DECLARE
    CURSOR get_prod_data IS
        SELECT libelle, prix_unit FROM  produit
        ORDER  BY prix_unit;
BEGIN
  FOR c1_rec IN get_prod_data
   LOOP
     DBMS_OUTPUT.PUT_LINE('produit :'|| c1_rec.libelle
                                || 'prixunitaire :'|| c1_rec.prix_unit) ;
   END LOOP ;
END ;
```

Les curseurs

- Les attributs d'un curseur
  - L'attribut *%FOUND*
    - Prend la valeur *TRUE* si le dernier ordre SQL a ramené au moins une ligne.
    - Utilisé avec la syntaxe
      - *Nom_curseur%FOUND* pour un curseur explicite
      - *SQL%FOUND* pour un curseur implicite

    - **Exemple** :

    Avec l'exemple précédent et les déclarations suivantes :

    lib_prod produit.libelle%TYPE;

    Prix_prod produit.prix_unit%TYPE;

Les curseurs

```
BEGIN
   OPEN get_prod_data;
      LOOP
      FETCH get_prod_data INTO lib_prod, prix_prod
      IF get_prod_data%FOUND THEN
      DBMS_OUTPUT.PUT_LINE('libelle:'|| lib_prod||' -prix:'
        ||prix_prod);
      ELSE EXIT;
      END IF;
      END LOOP
   CLOSE get_prod_data;
END;
/
```

Les curseurs

- L'attribut *%NOTFOUND*
  - Prend la valeur *TRUE* Si le dernier ordre SQL n'a ramené aucune ligne.
  - Utilisé avec la syntaxe
    - *Nom_curseur%NOTFOUND* pour un curseur explicite.
    - *SQL%NOTFOUND* pour un curseur implicite.
  - **Exemple** :

    mettre à jour la ville d'un étudiant et afficher un message si cet étudiant n'existe pas

Les curseurs

```
SQL> ACCEPT prod_ref PROMPT 'entrer une référence : '
entrer une référence : P1
SQL> BEGIN
UPDATE produit
   SET prix_unit=200
   WHERE ref_produit ='&prod_ref';
IF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('Ce produit n’existe pas') ;
END IF;
END;
```

Les curseurs

- L'attribut *%ROWCOUNT*
  - est de type numérique,
  - contient le nombre de lignes ramenées par le dernier ordre SQL,
  - utilisé avec la syntaxe
    - *Nom_curseur%ROWCOUNT* pour un curseur explicite.
    - *SQL%ROWCOUNT* pour un curseur implicite.

  - **Exemple** :

    Diminuer les prix des produits ayant un prix supérieur à 20 et afficher le nombre des produits modifiés..

Les curseurs

```
SET SERVEROUTPUT ON
BEGIN
UPDATE produit
   SET  prix_unit = prix_unit - 2
   WHERE prix_unit >= 20;
DBMS_OUTPUT.PUT_LINE('Nombre de produits
                                    modifiés:' || SQL%ROWCOUNT);
END;
```

Les curseurs

- L'attribut *%ISOPEN*
  - Est de type booléen,
  - utilisé pour déterminer si un curseur est ouvert ou fermé,
  - utilisé avec la syntaxe
    - *Nom_curseur%ISOPEN* pour un curseur explicite.
    - *SQL%ISOPEN* pour un curseur implicite.
    - **Remarque** : toujours FALSE pour les curseurs implicites

Les exceptions

- Les sources d'erreurs lors de l'exécution d'une application sont multiples :
  - Erreur du code,
  - erreur système,
  - violation d'une règle Oracle,
  - dépassement des limites du système d'exploitation.
- Chaque erreur Oracle générée par le noyau a un code erreur.

Les exceptions

- En PL/SQL, les erreurs de programme sont gérées au moyen de gestionnaires d'exceptions.
- Lorsqu'une erreur survient pendant l'exécution d'un programme, ce dernier détermine si on a anticipé le problème à l'aide d'un code.
- Oracle dispose d'un certain nombre de gestionnaires d'exceptions prédéfinis qu'on peut utiliser.
- Une exception permet de gérer une erreur pour laquelle oracle n'a pas prévu de gestionnaire.

Les exceptions

- Un nom d'exception est :
  - Soit prédéfini en interne par le noyau Oracle,
  - soit défini par l'utilisateur. Il peut être associé à un code d'erreur.
- Le développeur peut également définir ses propres erreurs qu'il doit déclencher explicitement et auxquelles il attribue un nom personnalisé.

Les exceptions

- Deux méthodes pour gérer les erreurs:
  - Si le nom d'exception existe

```
BEGIN
   EXCEPTION
      WHEN Nom_exception THEN
                -- Instructions;
END;
/
```

Les exceptions

- Si le nom d'exception n'existe pas:

```
BEGIN
  EXCEPTION
        WHEN OTHERS THEN
            IF  SQLCODE =Numero_code1  THEN
              -- Instructions;
            END IF;
            IF  SQLCODE=Numero_code2  THEN
              -- Instructions;
            END IF;
  END;
```

Les exceptions

- La commande *RAISE* interrompt l'exécution normale du bloc PL/SQL et transfère la suite des opérations au gestionnaire d'exception.

Les exceptions

- **Exemple** : Gérer un seuil de stock:

```
DECLARE
   stock_alert  EXCEPTION ;
   stock_seuil  NUMBER(4);
   nombre_stock  NUMBER(4);
BEGIN
   IF nombre_stock< stock_seuil THEN RAISE stock_alert  ;
   END IF ;
EXCEPTION
   WHEN stock_alert THEN …
END;
```

Les exceptions

- Quelques erreurs qu'oracle peut aider à gérer

Exception	Description	N° d'erreur
NO_DATA_FOUND	Est produite lorsqu'une instruction *SELECT INTO* ne retourne aucune ligne.	ORA-01403
TOO_MANY_ROWS	Est produite lorsqu'une instruction *SELECT INTO* retourne plusieurs lignes.	ORA-01422
DUP_VAL_ON_INDEX	Est produite lorsqu'un *INSERT* tente d'insérer une valeur en double dans une colonne sur laquelle une contrainte d'index unique a été définie.	ORA-00001

Les exceptions

- Quelques erreurs qu'oracle peut aider à gérer :

Exception	Description	N° d'erreur
VALUE_ERROR	Est produite lorsqu'on tente de placer une valeur dans une variable de type non compatible.	ORA-06502
OTHERS	Sert à intercepter n'importe quelle erreur qui n'est pas prise en charge par un gestionnaire spécifique. Elle doit apparaître en dernier dans la section des exceptions, car oracle ne traitera plus aucune exception après celle-ci.	

Les exceptions

- **Exemple**

```
DECLARE
      prod_row  produit%ROWTYPE ;
BEGIN
     SELECT * INTO prod_row FROM  produit
                ORDER BY  nom;
EXCEPTION
      WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('aucune donnée dans' ||'cette
                                        table Essayez a nouveau !') ;
    WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE(' il y a un probleme') ;
END ;
```

Les exceptions

- *PRAGMA EXCEPTION_INIT*
  - Est une directive compilée qui demande au compilateur d'associer le nom d'une exception avec le numéro d'une erreur Oracle de la façon suivante:

PRAGMA EXCEPTION_INIT(Nom_exception, Numero_erreur_Oracle);

Les exceptions

- **Exemple**: Déclarer l'exception Privilges_insuffisants et l'associer à l'erreur -1031:

```
DECLARE
   Privileges_insuffisants EXCEPTION;
   PRAGMA EXCEPTION_INIT(Privileges_insuffisants,-1O31);
BEGIN
EXCEPTION
    WHEN Privilges_insuffisants THEN
              Traitements;
END ;
```

Les exceptions

- Il est possible de créer ses propres erreurs avec numéro de code et message associé en utilisant la syntaxe suivante :

  RAISE_APPLICATION_ERROR(numéro_erreur, message)

  où numéro_erreur est compris entre -20000 et -20999.

Les exceptions

- **Exemple :**

```
SQL> ACCEPT pdt_num
DECLARE
lib_pdt  produit.libelle%TYPE;
BEGIN
   SELECT libelle INTO lib_pdt
        FROM produit
       WHERE ref_produit =&pdt_num;
   IF lib_pdt = ‘pantalon enfant' THEN
     RAISE_APPLICATION_ERROR( -20200, 'C’EST pantalon
                                                 enfant! ');
   ELSE
       DBMS_OUTPUT.PUT_LINE(lib_pdt);
  END IF ;
END;
```

Les exceptions

- La fonction *SQLCODE* retourne le numéro de l'erreur Oracle.
  - Ce nombre est négatif sauf pour l'erreur *NO_DATA_FOUND* : (*SQLCODE* = +100).
- La fonction *SQLERRM* retourne le texte du message d'erreur.
  - Ce message commence avec le code Oracle de l'erreur.

Les exceptions

- Exemple : afficher les messages pour les numéros d'erreurs allant de -20 à -30

```
DECLARE
Message_erreur VARCHAR2(1000);
BEGIN
   FOR Numero_erreur IN 20.. 30 LOOP
      Message_erreur : = SQLERRM( - Numero_erreur);
      DBMS_OUTPUT.PUT_LINE(message_erreur);
   END LOOP;
END;
```

Les traitements stockés

- Les traitements sont définis avec le langage PL/SQL et sont stockés dans les tables du dictionnaire de données.
- On peut créer quatre types de programmes stockés : des procédures, des fonctions, des paquetages et des triggers.

Les traitements stockés

- Les avantages des traitements stockés :
  - Réutilisation des programmes : toutes les applications peuvent accéder au programme.
  - Permettre leur partage entre des utilisateurs et des applications.
  - Faciliter la gestion de la confidentialité : donner le droit de lancer un traitement sans donner le droit d'accès direct aux données.

Les procédures

- Les procédures stockées sont des programmes entiers qui peuvent être exécutés d'une façon indépendante ou à partir d'un autre programme,
- Remarque : Bien que les fonctions et les paquetages soient eux aussi stockées dans la base de données, habituellement seules les procédures sont dites stockées.

Les procédures

- Syntaxe de création d'une procédure :

```
CREATE [OR REPLACE] PROCEDURE nom_procedure
[(nom_argument1 [IN|OUT] type, (nom_argument2 {IN|OUT}
   type,…)]
AS | IS
BEGIN
[partie déclaration]
BEGIN
...
[EXCEPTION...]
END;
END [nom procedure];
```

Les procédures

- Avec:
  - Partie déclaration est similaire à celle d'un bloc PL/SQL.
  - *IN* : argument en entrée, non modifié par la procédure.
  - *OUT* : argument en sortie, peut être modifié par la procédure, transmis au programme appelant.
  - *IN OUT* : à la fois en entrée et en sortie.

Les procédures

- **Exemple1** :

```
CREATE OR REPLACE PROCEDURE list_etudiant IS
BEGIN
  DECLARE
      pdt_count  NUMBER ;
      CURSOR get_pdt_data IS
            SELECT libelle, prix_unit FROM  etudiant
             ORDER BY nom;
   BEGIN
     FOR c1_rec IN get_etud_data
      LOOP
       DBMS_OUTPUT.PUT_LINE('produit :'|| c1_rec.libelle || '
                                                prix :'||  c1_rec.prix_unit) ;
      END LOOP ;
    END ;
END;
```

Les procédures

- Remarques :
  - Il peut être utile de décrire une procédure avant de l'exécuter à l'aide de la commande *DESCRIBE*

    DESCRIBE nom_procedure
  - Exécution d'une procédure stockée Sous sql*Plus :

    EXECUTE nom_procedure [(liste arguments)]
  - Appel d'une procédure dans un programme PL/SQL :

    nom_procedure [(liste arguments effectifs)];

Les procédures

- Recherche d'une procédure dans la base

```
SELECT object_name
FROM user_objects
WHERE object_type='PROCEDURE'
```

Les procédures

- Recherche du code source d'une procédure dans la base :

```
SELECT text
FROM user_source
WHERE name='nom_procedure'
ORDER BY LINE ;
```

- Suppression d'une procédure :

```
DROP PROCEDURE nom_procedure;
```

Les fonctions

- Sont aussi des objets stockés dans la base,
- Une fonction retourne un resultat
- Il est possible de les utiliser dans des instructions SQL.

Les fonctions

- Syntaxe de création d'une fonction :

```
CREATE OR REPLACE FUNCTION nom_function
[(liste parametres)] RETURN type_val  {IS|AS}
[partie déclaration]
BEGIN
  {contenu du bloc PL}
   RETURN variable_a_retourner;
[EXCEPTION...]
END [nom_function];
```

Les fonctions

- Avec:
  - Partie déclaration est similaire à celle d'un bloc PL/SQL.
  - *Type_val :* le type de la valeur retournée par la fonction.
  - *Liste parametres* : similaire à une procédure.

Les fonctions

- Exemple 1 :

```
CREATE OR REPLACE FUNCTION count_produit
RETURN NUMBER
IS
BEGIN
     i  NUMBER ;
    CURSOR get_pdt_data IS
                SELECT COUNT (*) FROM  produit;
BEGIN
OPEN get_pdt_data;
FETCH get_pdt_data INTO i;
RETURN i;
END;
END;
```

Les fonctions

- Exemple 2 :

```
CREATE OR REPLACE FUNCTION lib_produit(p_no IN produit.
   Ref_produit% TYPE )
   RETURN char(30)
 IS
BEGIN
DECLARE
Pdt_libelle produit.libelle%type ;
BEGIN
SELECT libelle INTO Pdt_libelle FROM produit
  WHERE ref_produit=p_no ;
RETURN (Pdt_libelle  ;
END;
END;
```

Les fonctions

- Remarques :
  - Exécution d'une fonction Sous sql*Plus :

    SELECT nom_fonction [(liste arguments)] FROM DUAL;
  - Appel d'une fonction dans un programme PL/SQL :

    Variable:=nom_fonction [(liste arguments effectifs)];

Les fonctions

- Recherche d'une fonction dans la base :

```
SELECT object_name
FROM user_objects
WHERE object_type='FUNCTION'
```

Les fonctions

- Recherche du code source d'une fonction dans la base :

```
SELECT text
FROM user_source
WHERE name='nom_function'
ORDER BY LINE ;
```

- Suppression d'une fonction :

```
DROP FUNCTION nom_fonction;
```